

Rhilo Sotto

30 September 2024

Assignment 1

Modules

One-Bit Full-Adder (Gate Level)

```
`timescale 1ns / 1ps

// gate level full adder module
module OneBitFullAdderGate(input A,input B,input Cin,
    output Cout,output S);

    // intermediary signals
    wire x1, x2, x3;

    xor (x1, A, B);
    xor (S, x1, Cin);

    and (x2, x1, Cin);
    and (x3, A, B);
    or (Cout, x2, x3);

endmodule
```

One-Bit Full-Adder (Dataflow Level)

```
`timescale 1ns / 1ps
```

```
// dataflow level full adder module
```

```
module OneBitFullAdderDataflow(input A,input B,input Cin,  
    output Cout,output S);
```

```
    assign S = A ^ B ^ Cin;
```

```
    assign Cout = ((A ^ B) & Cin) | (A & B);
```

```
endmodule
```

Four-Bit Full-Adder (Dataflow Level)

```
`timescale 1ns / 1ps

// dataflow level four bit full adder module
module FourBitFullAdderDataflow(input [3:0] A,input [3:0] B,input Cin, input Sel,
    output [4:1] Cout,output [3:0] S,output OVF);

    // if Cin/Sel is 0/1, multiplexer for compliment
    wire [3:0] Bn;
    assign Bn = (Sel == 0) ? B : ~B;

    assign S[0] = A[0] ^ Bn[0] ^ Cin;
    assign Cout[1] = ((A[0] ^ Bn[0]) & Cin) | (A[0] & Bn[0]);

    assign S[1] = A[1] ^ Bn[1] ^ Cout[1];
    assign Cout[2] = ((A[1] ^ Bn[1]) & Cout[1]) | (A[1] & Bn[1]);

    assign S[2] = A[2] ^ Bn[2] ^ Cout[2];
    assign Cout[3] = ((A[2] ^ Bn[2]) & Cout[2]) | (A[2] & Bn[2]);

    assign S[3] = A[3] ^ Bn[3] ^ Cout[3];
    assign Cout[4] = ((A[3] ^ Bn[3]) & Cout[3]) | (A[3] & Bn[3]);

    assign OVF = Cout[4] ^ Cout[3];

endmodule
```

Two-One Multiplexer (Used in Structural Level Four-Bit Full-Adder)

```
`timescale 1ns / 1ps

// 2:1 multiplexer
module TwoOneMultiplexer(input In0,input In1,input Sel,
    output Out);

    // intermediary wires
    wire SelNot;
    wire x1, x2;

    not (SelNot, Sel);

    and (x1, In0, SelNot);
    and (x2, In1, Sel);

    or (Out, x1, x2);

endmodule
```

Four-Bit Full-Adder (Structural Level)

```
`timescale 1ns / 1ps
```

```
// structural level four bit full adder module
```

```
module FourBitFullAdderStructural(input [3:0] A,input [3:0] B,input Cin,input Sel,  
    output [4:1] Cout,output [3:0] S,output OVF);
```

```
    wire [3:0] Bn;
```

```
    // instantiate array of multiplexers
```

```
    TwoOneMultiplexer MUX0(.Out(Bn[0]), .In0(B[0]), .In1(!B[0]), .Sel(Sel)),  
        MUX1(.Out(Bn[1]), .In0(B[1]), .In1(!B[1]), .Sel(Sel)),  
        MUX2(.Out(Bn[2]), .In0(B[2]), .In1(!B[2]), .Sel(Sel)),  
        MUX3(.Out(Bn[3]), .In0(B[3]), .In1(!B[3]), .Sel(Sel));
```

```
    // instantiate array of one bit full adders
```

```
    OneBitFullAdderGate FA0(.A(A[0]), .B(Bn[0]), .Cin(Cin), .S(S[0]), .Cout(Cout[1])),  
        FA1(.A(A[1]), .B(Bn[1]), .Cin(Cout[1]), .S(S[1]), .Cout(Cout[2])),  
        FA2(.A(A[2]), .B(Bn[2]), .Cin(Cout[2]), .S(S[2]), .Cout(Cout[3])),  
        FA3(.A(A[3]), .B(Bn[3]), .Cin(Cout[3]), .S(S[3]), .Cout(Cout[4]));
```

```
    xor (OVF, Cout[4], Cout[3]);
```

```
endmodule
```

Eight-Bit Full-Adder (Structural Level)

```
`timescale 1ns / 1ps
```

```
// structural level eight bit full adder module
```

```
module EightBitFullAdderStructural(input [7:0] A,input [7:0] B,input Cin, input Sel,  
    output [8:1] Cout,output [7:0] S,output OVF);
```

```
    wire carry;
```

```
    // four least significant bits
```

```
    FourBitFullAdderDataflow FBA0(.A(A[3:0]), .B(B[3:0]), .Cin(Cin), .Sel(Sel), .Cout(Cout[4:1]),  
    .S(S[3:0]));
```

```
    // four most significant bits
```

```
    FourBitFullAdderStructural FBA1(.A(A[7:4]), .B(B[7:4]), .Cin(Cout[4]), .Sel(Sel),  
    .Cout(Cout[8:5]), .S(S[7:4]), .OVF(OVF));
```

```
endmodule
```

Testbenches

One-Bit Full-Adder

```
`timescale 1ns / 1ns
```

```
module tb_OneBitFullAdder;
```

```
    // inputs
```

```
    reg a, b, ci;
```

```
    // outputs
```

```
    wire co_g, s_g;
```

```
    wire co_d, s_d;
```

```
    // instantiate DUTs
```

```
    OneBitFullAdderGate FA0( .A(a), .B(b), .Cin(ci), .Cout(co_g), .S(s_g) );
```

```
    OneBitFullAdderDataflow FA1( .A(a), .B(b), .Cin(ci), .Cout(co_d), .S(s_d) );
```

```
    initial begin
```

```
        a = 1'b0; b = 1'b0; ci = 1'b0;
```

```
        #10; a = 1'b1;
```

```
        #10; b = 1'b1;
```

```
        #10; ci = 1'b1;
```

```
        #10; b = 1'b0;
```

```
        #10;
```

```
        #1 $finish;
```

```
    end
```

```
endmodule
```

Four-Bit Full-Adder

```
`timescale 1ns / 1ns
```

```
module tb_FourBitFullAdder;
```

```
    // inputs
```

```
    reg [3:0] a, b;
```

```
    reg ci, sel;
```

```
    // outputs
```

```
    wire [4:1] co_d, co_s;
```

```
    wire [3:0] s_d, s_s;
```

```
    wire ovf_d, ovf_s;
```

```
    // instantiate DUTs
```

```
    FourBitFullAdderDataflow FBA0( .A(a), .B(b), .Cin(ci), .Sel(sel), .Cout(co_d), .S(s_d),  
.OVF(ovf_d));
```

```
    FourBitFullAdderStructural FBA1( .A(a), .B(b), .Cin(ci), .Sel(sel), .Cout(co_s), .S(s_s),  
.OVF(ovf_s));
```

```
    initial begin
```

```
        // addition
```

```
        a = 4'b0000; b = 4'b0000; ci = 1'b0; sel = 1'b0;
```

```
        #10; a = 4'b1100; b = 4'b0101; // a = -4, b = 5
```

```
        #10; a = 4'b0111; b = 4'b0111; // a = 7, b = 7
```

```
        #10; a = 4'b1000; b = 4'b1000; // a = -8, b = -8
```

```
        #10; a = 4'b0111; b = 4'b1000; // a = 7, b = -8
```

```
        // subtraction
```

```
        #10; a = 4'b0000; b = 4'b0000; ci = 1'b1; sel = 1'b1;
```

```
        #10; a = 4'b1100; b = 4'b0101; // a = -4, b = 5
```

```
        #10; a = 4'b0111; b = 4'b0111; // a = 7, b = 7
```

```
        #10; a = 4'b1000; b = 4'b1000; // a = -8, b = -8
```

```
        #10; a = 4'b0111; b = 4'b1000; // a = 7, b = -8
```

```
        #10;
```

```
        #1 $finish;
```

```
    end
```

```
endmodule
```


Eight-Bit Full-Adder

```
`timescale 1ns / 1ns
```

```
module tb_EightBitFullAdder;
```

```
    // inputs
```

```
    reg [7:0] a, b;
```

```
    reg ci, sel;
```

```
    // outputs
```

```
    wire [8:1] co;
```

```
    wire [7:0] s;
```

```
    wire ovf;
```

```
    // instantiate DUTs
```

```
    EightBitFullAdderStructural EBA0( .A(a), .B(b), .Cin(ci), .Sel(sel), .Cout(co), .S(s), .OVF(ovf));
```

```
    initial begin
```

```
        // addition
```

```
        a = 8'b00000000; b = 8'b00000000; ci = 1'b0; sel = 1'b0;
```

```
        #10; a = 8'b11110000; b = 8'b00001000; // a = -16, b = 8
```

```
        #10; a = 8'b00001111; b = 8'b00001111; // a = 15, b = 15
```

```
        #10; a = 8'b11110111; b = 8'b11110001; // a = -9, b = -15
```

```
        #10; a = 8'b11110000; b = 8'b11110000; // a = -16, b = -16
```

```
        // subtraction
```

```
        #10; a = 8'b00000000; b = 8'b00000000; ci = 1'b1; sel = 1'b1;
```

```
        #10; a = 8'b11110000; b = 8'b00001000; // a = -16, b = 8
```

```
        #10; a = 8'b00001111; b = 8'b00001111; // a = 15, b = 15
```

```
        #10; a = 8'b11110111; b = 8'b11110001; // a = -9, b = -15
```

```
        #10; a = 8'b11110000; b = 8'b11110000; // a = -16, b = -16
```

```
        #10;
```

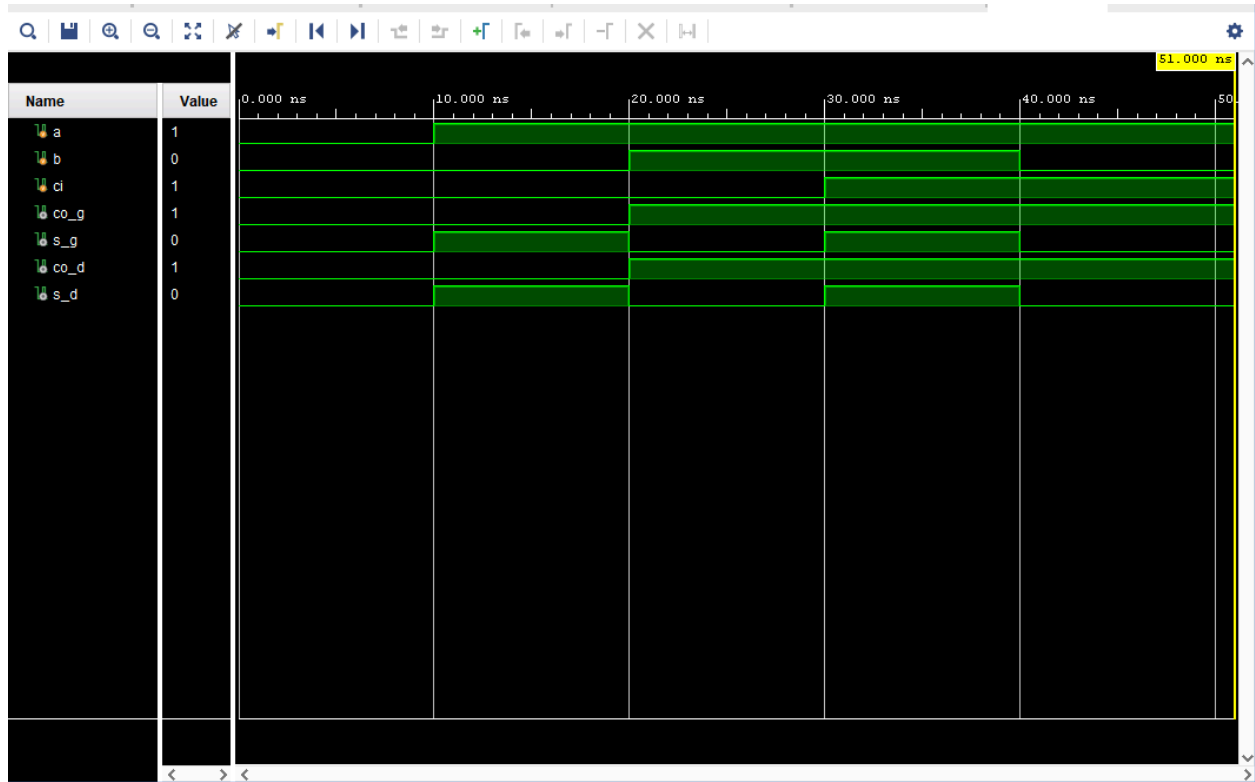
```
        #1 $finish;
```

```
    end
```

```
endmodule
```

Waveforms

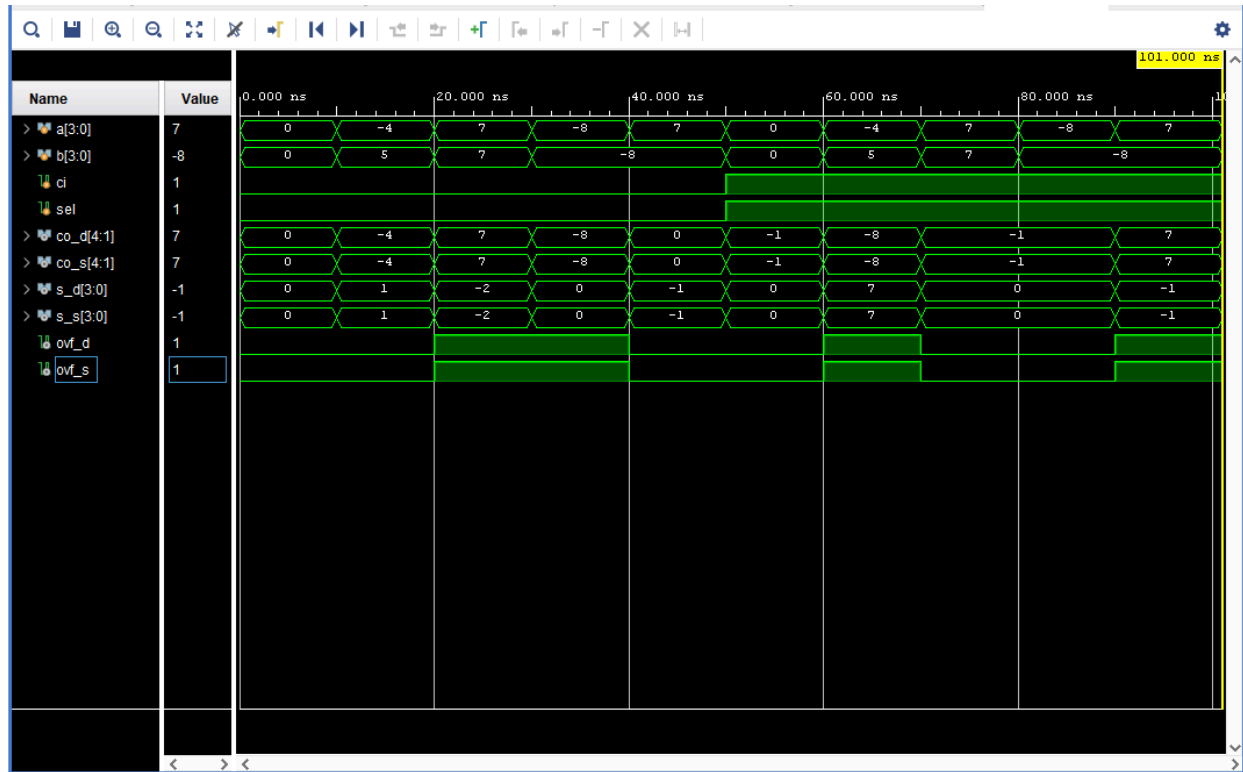
One-Bit Full-Adder



Corresponds to gate level (_g) and dataflow level (_d) one-bit full-adders and one-bit full-adder testbench.

The output is as expected, with co_ as the carry bits, and s_ as the sum.

Four-Bit Full-Adder



Corresponds to dataflow level (_d) and structural level (_s) four-bit full-adders and four-bit full-adder testbench.

The output is as expected, with co_ as the carry bits, s_ as the sum, and ovf_ as the overflow.

Eight-Bit Full-Adder

